

Following Breadcrumbs in Code

What Accidentally Committed Ad-Hoc Logs Reveal about Developer Comprehension

Yi-Hung Chou · Boyuan Jiang · Vidit Jain · Yiyang Min · April Yi Wang · James A. Jones

Received: date / Accepted: date

Abstract Developers frequently insert temporary print or log statements—ad-hoc logs—to better understand program behavior at runtime, particularly when facing unexpected issues or complex control flows. Despite being a nearly universal practice, systematic study has been limited because these logs are ephemeral: they usually remain only in local environments and are removed before code is committed, making them difficult to capture. In this work, we addressed this challenge by mining accidental commits where developers unintentionally left ad-hoc logs and later deleted them, and by analyzing live-streamed programming sessions to observe their use in practice. Using these methods, we constructed a large dataset across three major programming languages (Java, JavaScript, and Python), enabling the large-scale investigation of logging practices. Our analysis reveals both common and language-specific patterns in where and how developers rely on ad-hoc logs. Across languages, ad-hoc logs tend to appear in program regions that are harder to reason about at runtime. We also identify distinctive language-level patterns, such as frequent use in asynchronous and callback functions in JavaScript and in thread-related classes in Java. In addition, functions containing ad-hoc logs generally have higher cyclomatic complexity than the overall function population. Production logs show a similar association, consistent with logging serving as a means of observing runtime behavior in structurally complex functions. Together, these findings provide empirical insight into developers’ runtime comprehension practices and offer a valuable dataset for researchers and tool builders seeking to better support debugging and logging.

Keywords Printf Debugging · Program Comprehension

Y. Chou¹ · B. Jiang¹ · V. Jain¹ · Y. Min² · A. Wang³ · J. A. Jones¹

¹University of California, Irvine, USA

²Amazon, Toronto, Canada

³ETH Zürich, Switzerland

E-mail: {yihungc1, boyuanj, viditj3, jajones}@uci.edu, rogermyy@amazon.com, april.wang@inf.ethz.ch

1 Introduction

Logging statements in software development generally serve two distinct purposes. (1) The first concerns statements that remain in the codebase after deployment, commonly referred to as **production logging statements**. These persistent statements are used to monitor run-time behavior and provide diagnostic information, thereby supporting developers in maintaining and troubleshooting software in production. (2) The second, and arguably less recognized yet commonly seen, use of `printf` statements involves their temporary insertion during development. In this context, developers rely on them as lightweight tools for debugging, testing, or understanding program behavior. Such statements, or **ad-hoc logs**, are typically removed prior to deployment, and this practice is often referred to as **printf debugging**.

Logging statements, particularly those that are meant to survive into production, have been extensively studied in the software engineering community, as shown in the survey conducted by Gholamian and Ward (2022) and Chen and Jiang (2022). For example, Fu et al. (2014) investigated where developers place their production logs for monitoring purposes, while Li et al. (2023) conducted manual inspections and surveys to evaluate the readability of logging statements. Building on such motivations, researchers have also proposed methods and tools to automate the improvement of production logging practices. For instance, Zhu et al. (2015) developed tools to support developers in making informed decisions when inserting production logs.

However, to the best of our knowledge, the second type of logs—ad-hoc logs—remains largely understudied in the software engineering literature. Prior observational studies report that developers use ad-hoc logs as one of the most common strategies for program comprehension and debugging (Alaboudi and LaToza, 2019, 2023; Jiang et al., 2023; Beller et al., 2018; Siegmund et al., 2014). Despite their recognized importance, the transient and local nature of these ad-hoc logs has constrained the community’s ability to systematically study and understand them.

Although the collection of transient and localized ad-hoc log statements may initially appear infeasible, we argue that two inherent properties of such logs enable their systematic and large-scale identification. First, ad-hoc logs are, by design, not intended to remain in production software. Nevertheless, developers are not immune to errors, and prior empirical studies have shown that inadvertent commits of unintended code artifacts are common in practice Wen et al. (2020); Meli et al. (2019). As a result, when ad-hoc logs are mistakenly introduced into a repository, developers typically issue subsequent corrective commits to remove these statements, thereby leaving recoverable traces. Second, qualitative evidence reported in Chou et al. (2025) indicates that ad-hoc logs are most often implemented through lightweight, native output functions (e.g., `System.out.println`) rather than standardized and comparatively heavyweight logging frameworks such as `log4j`. This implementation choice not only distinguishes ad-hoc logs from production-level logging but also makes their detection more tractable. By capitalizing on these proper-

Python: lenstronomy/lenstronomy**Message:** removing debugging print statements

```
if check_convention and not self.ignore_observed_positions:
    kwargs_lens = self._convention(kwargs_lens)
- print('I am leaving ray shooting partial')
```

Java: liquibase/liquibase**Message:** DAT-3587 Remove println

```
protected void doMigration() throws Exception {

    if (COMMANDS.HELP.equalsIgnoreCase(command)) {
-       System.out.println("HERE!!!!!!!!!!!!!!!!!!!!!!");
        printHelp(System.err);
    }
}
```

JavaScript: ellismarkf/lark**Message:** remove console logs

```
option.destinationArrivalHour = destinationHour;
option.destinationArrivalMins = destinationMins;
- console.log('is this code running?');
```

Fig. 1 Examples of accidentally committed ad-hoc logs collected in our dataset.

ties, our research leverages the occurrence of ad-hoc logs within corrective commits as a serendipitous opportunity to address a critical research gap: the large-scale collection and analysis of ad-hoc log statements as a means to infer developers' comprehension processes and debugging challenges during software development.

This study sheds light on the subject by examining accidental commits in version-control systems like GitHub, where developers explicitly remove ad-hoc logs, as noted in commit messages (e.g., “remove console.log” or “delete print”). Figure 1 provides examples of ad-hoc logs and the corresponding commit messages. To gain deeper insight into how these logs are used, we also conducted a qualitative analysis of 36 hours of developers' live-stream sessions collected by Alaboudi and LaToza (2019). Unlike the GitHub commits, which only capture mistakenly preserved logs, the live-stream data reveals all instances of temporary logging and their practical use. This comparison helps us better understand the situations and comprehension challenges in which developers rely on these logs. In short, the GitHub data offers broad coverage and scalability, while the live-stream data provides depth and nuanced qualitative understanding.

Our analysis uncovered several noteworthy patterns across three programming languages—Java, JavaScript, and Python. One recurring practice was that developers frequently structured their console outputs in ways that allowed them to quickly identify and compare program states or execution flows in real time. We also observed that ad-hoc logging was disproportionately concentrated in certain types of code locations. For instance, in JavaScript, developers inserted print statements into asynchronous and callback functions more often than would be expected given the overall distribution of such functions in the dataset, suggesting that these contexts demanded additional logging to clarify their behavior. Similarly, in Java, developers placed print statements into thread-related classes more than the overall distribution of such classes in the sample dataset. Furthermore, across three programming languages, ad-hoc logs were used more frequently in functions with high Cyclomatic Complexity than in simpler ones, highlighting their role in aiding comprehension of challenging code segments.

Our study makes the following contributions through extending work in Chou et al. (2025): (1) We extended the empirical study for understanding ad-hoc logs across three languages, guided by research questions informed by prior literature. (2) We analyzed developers’ live-coding sessions from Alaboudi and LaToza (2019, 2023) to gain insights into their ad-hoc logging behaviors, which provides qualitative context for understanding the motivations and ad-hoc log-authoring practices. (3) We introduced a novel mining method that captures data previously considered inaccessible or hidden in local environments, such as ad-hoc logs, by leveraging developer mistakes. (4) We collected and published JavaScript, Java, and Python commits from GitHub Archive Grigorik (2015) and Google Cloud BigQuery Google (2025), containing 882,438 removed logs in total, along with corresponding metadata from the GitHub APIs GitHub, Inc. (2025).

2 Background & Motivation

In this section, we examine two areas of related research. First, we review prior work on how developers comprehend the dynamic behavior of software, which motivates our study. Second, we discuss research on production logging, which informs our methodology and shapes our research questions.

2.1 Comprehending Dynamic Behaviors of Software

Developers must construct accurate mental models of the systems they build, yet this task is often challenging and time-consuming. Gilmore (1991) conducted studies of over 80 developers performing debugging tasks and noted that “the success of experts at debugging is not due to better debugging skills, but rather to better comprehension.” Empirical studies report that developers spend between 35% and 58% of their time on program comprehension

(Ko et al. (2006); Xia et al. (2018)). Among such comprehension challenges, understanding dynamic behaviors—such as evolving program states or complex execution flows—remains particularly demanding. To gain insights into run-time behavior, developers frequently insert logging statements, rerun the program, or interact with user interfaces (Maalej et al. (2014); Beller et al. (2018)). Even with the emergence of artificial intelligence, one of the most time-consuming and challenging parts of the work for developers is to audit and understand the generated code as found in Gordon et al. (2023); Bird et al. (2023).

Despite its importance, evaluating developers’ comprehension remains a challenging task. Researchers commonly rely on small participant groups in controlled or within-subject studies (Xia et al. (2018)), distribute surveys that trade qualitative nuance for quantitative breadth (Beller et al. (2018); Maalej et al. (2014)), or employ proxy measures such as cognitive load or Cyclomatic Complexity (Rilling and Klemola (2003)). As noted by Ko et al. (2015), studies involving human participants demand careful recruitment and management, which constrains scalability even though they provide rich qualitative insights. In contrast, proxies such as cognitive complexity—derived from program structure rather than grounded in empirical studies of developer cognition—scale more easily but may fail to reflect the actual cognitive challenges developers encounter, as argued in Shepperd (1988); Lenarduzzi et al. (2023). Motivated by both the difficulty of recruiting human participants and the limitations of complexity metrics, our work explores the use of empirical data—specifically ad-hoc log placements—to approximate developers’ comprehension challenges.

2.2 Tracking Back in the Forest: Ad-hoc vs. Production Logs

To properly situate the reader, we review and contrast the existing research on production logs to our new work on ad-hoc logs. When navigating the dense forest of a running program, developers leave different kinds of markers to trace their path. Production logs are like stones or trail markers—carefully placed and meant to endure, guiding future maintainers along established routes.

Research on production logs has been extensive, with much of it emphasizing their role in long-term system monitoring and maintenance. We summarize this body of work across four perspectives: **Why**, **How**, **Where**, and **What**.

Why do developers write logs?: Li et al. (2021) surveyed 66 developers and analyzed 223 logging-related issue reports, identifying four primary benefits of logging: troubleshooting, execution tracking, comprehension support, and bookkeeping. In the context of embedded software engineering, Yang et al. (2021) interviewed 28 developers and found that production logs are mainly used for problem localization and performance improvement, rather than for reverse engineering or understanding existing systems.

How do developers log?: Li et al. (2017) showed that developers’ choices of logging *levels* are strongly influenced by existing logging statements and surrounding block types. Jiang et al. (2023) also examined challenges in log

authoring, identifying issues such as disorganized log output, insufficiently informative visualizations, frequent context loss due to view switching, and trade-offs in crafting and interpreting logs.

Where do developers place logs in code?: The placement of production logs has also received significant attention. Fu et al. (2014) conducted a mixed-methods study at Microsoft, combining code analysis with interviews, and identified five categories of log locations. Building on this work, Li et al. (2020) analyzed seven large open-source Java systems and reported seven categories of log placement and their distributions. Both studies laid the foundation for log-location recommendation systems, which have since been refined through learning-based approaches, such as works in Mastropaolo et al. (2022); Zhu et al. (2015); Li (2020).

What data do developers log?: The content of logging statements has received comparatively less attention. Liu et al. (2021) proposed an RNN-based tool to recommend which variables developers should log, though without deeper characterization of these variables. Jiang et al. (2023) observed that developers frequently rely on template strings and labels to maintain continuity between source code and log output.

In contrast, ad-hoc logs resemble breadcrumbs—dropped quickly, used once, and often discarded when their purpose has been served. Research on these “breadcrumbs” remains far more limited, likely due to their transient nature. Beller et al. (2018) highlighted their significance, showing that most developers continue to rely on simple print statements for debugging. This practical reliance has long been echoed by practitioners: Unix contributor Kernighan (1978) remarked that “the most effective debugging tool is still careful thought, coupled with judiciously placed print statements.” To date, however, only Jiang et al. (2023) have explicitly investigated ad-hoc logging through observational studies. Specifically, they examined six novice and six expert JavaScript programmers to inform the development of their tool, Log-It. Their work focuses on ad-hoc logging as a form of live interaction among developers, source code, and terminal output, with the goal of identifying usability challenges when working with logs during debugging sessions. Our study, in contrast, adopts a more objective and scalable approach by quantitatively analyzing ad-hoc logs in version control systems. Through this analysis, we infer the code locations, structural contexts, and comprehension challenges that lead developers to introduce ad-hoc instrumentation in real-world projects.

Taken together, studies of production logs provide methodological insights and conceptual frameworks that inform our own inquiry into these ephemeral traces, guiding both our research questions and approach.

3 Research Questions

Building on prior research that explores the dimensions of *Where*, *What*, *Why*, and *How*, we define the following research questions to guide the analysis of our collected data.

RQ1: *Where* in the codebase do developers place ad-hoc logs?

RQ2: *What* information do developers record in ad-hoc logs?

RQ3: *Why* do developers rely on ad-hoc logs?

RQ4: *How* do developers manipulate ad-hoc logs (e.g., insert, revise, or remove) to achieve their goals?

4 Dataset Curation

In this section, we describe the curation process for the two datasets that underpin our empirical study: (1) ad-hoc log statements collected from Java (JAVA), Python (PY), and JavaScript/TypeScript (JS) projects, together with associated metadata such as commit information, repository details, and before-and-after code; and (2) live-stream recordings of developers adding ad-hoc logs for debugging or program comprehension. For reproducibility, the full dataset and the crawling scripts are provided in Chou (2026b).

Collecting Accidentally Pushed Printf Statements. To collect printf statements that were later removed by developers, we first identified the targeted commits from Google BigQuery and GitHub Archive. The process began by extracting all events of type *PushEvent* from GitHub Archive. From these events, we extracted the commits whose messages matched the target regular expression, which was derived from qualitative insights and empirical tests as described below. To capture these accidental commits, we needed a generalizable way of identifying commits that involved the removal of ad-hoc logging. Our solution was to reverse engineer a commit-message pattern that would reliably indicate such removals. We began by sampling 2,000 random repositories for each language. Within each repository, we inspected commits to locate those where developers had actually removed print or logging statements from the code. From this filtered set, we focused on the commit messages themselves and used embeddings to cluster them. We then selected 100 representative (centroid) messages and analyzed their common linguistic structure. Based on this analysis, we constructed a regular expression that could capture the recurring patterns found in those messages. The resulting regular expression thus reflects an empirical generalization of how developers typically describe ad-hoc log removals in commit messages, making it possible to scale our detection across large datasets.

For fetching the specific commits, we relied on GitHub’s event stream rather than the API due to GitHub’s API rate limits. We collected *PushEvents* spanning from February 12, 2011 to January 11, 2020. These events sometimes contained commits going back as far as 2007. To extend coverage, we later added a supplementary sample of commits obtained via Google BigQuery, reaching up to October 1, 2024. In this case, the BigQuery SQL query selected only the first commit from each *PushEvent* in the GitHub Archive and applied the same regular expression filters. For JAVA & PY: We used Google BigQuery to fetch the *PushEvents* from 2015 to 2022. In this dataset, we se-

lected all commits in each *PushEvent*, with a maximum 20 (as it is capped by the design of GitHub’s *PushEvent* query).

After retrieving the commits for all languages, we used GitHub APIs to collect the necessary data that is missing from the event metadata, such as actual files and their diffs in the commits, for subsequent analysis.

Collecting Production Logging Statements. To further investigate *where* ad-hoc logs are placed in codebases (RQ1), we contrast their locations with the locations of production logs. This comparison helps distinguish patterns that are specific to ad-hoc logging from patterns that may reflect logging practices more generally. Prior studies Fu et al. (2014); Li et al. (2020) provide useful insights into where developers place production logs. For example, Li et al. (2020) found that production logs in a corporate environment often appear in error-handling code or around logical branches, such as `if` statements. However, these distributions may not directly apply to our setting, because our ad-hoc log dataset is drawn primarily from personal and open-source projects.

To support a like-for-like comparison, we collected production logs from the same repositories in which we identified ad-hoc logs. Specifically, we shallow-cloned the repositories from which the filtered ad-hoc logging statements were collected and that remained publicly available. We then scanned the latest version of each repository to identify logging statements primarily intended for production use, such as `Log4j`¹ in Java, the `logging` module² in Python, and `Winston`³ in JavaScript. For each production log statement, we extracted the same attributes as for the ad-hoc logs, including the surrounding block type and the complexity of the enclosing function. This process yielded 390,191 production logs in Python, 163,033 in JavaScript, and 807,189 in Java.

Preparing Ground Truth for Comparison. To estimate the complexity and distribution of function types in public code, we curated three repository samples. The first consists of 1,000 JavaScript repositories created between 2023 and 2024, randomly drawn from 3,000 candidates retrieved via the GitHub Search API. The second and third each include 1,000 repositories, randomly sampled from GitHub Archive and GitHub BigQuery. For Java and Python, we randomly selected 2,000 repositories each from corpora where ad-hoc logs had been removed. Collectively, these samples provide a baseline for evaluating whether the placement of ad-hoc logs reflects substantive factors—such as function complexity, threading, and asynchrony—rather than convention alone. For example, the higher complexity observed in functions from which ad-hoc logs were removed may partly reflect the inherently greater structural complexity of those specific samples, and thus a related need for comprehension of runtime behaviors in such complex code.

Live-Streamed Videos. Live streaming is increasingly adopted by developers to increase visibility, support education, and foster community engagement, as

¹ <https://logging.apache.org/log4j/2.x/index.html>

² <https://docs.python.org/3/library/logging.html>

³ <https://github.com/winstonjs/winston>

noted in Kokinda and Rodeghero (2023). Such data enable unobtrusive observation of behaviors in natural settings, helping reduce observer bias (Mahtani et al. (2018)). Despite this potential, live streaming remains underexplored in software-engineering research. Progress has been made by curating and labeling live-streamed videos and by developing the *observer-dev.online* platform for software-engineering researchers as created by Alaboudi (2023). From this dataset, we used all 15 curated videos (total duration: 36:01:30) and conducted a thematic analysis, with both the dataset and our qualitative findings made publicly available (Chou (2026b)). Since little is known about why and how developers insert log statements, we relied on the segmentation of each video into debugging episodes (e.g., “*Testing the program and reading the outputs*”, “*Interacting with a file of code (Edit)*”) provided in Alaboudi and LaToza (2023). These segments allowed us to efficiently identify moments when developers began adding log statements to source code.

5 Data Analysis

In this section, we describe our data-analysis process for addressing the research questions above.

5.1 Analyzing Collected Ad-hoc Logs

To address RQ1 and RQ2, we prepared the dataset that contains three programming languages for analysis by conducting the following preprocessing steps:

Filtering: Some commits contained messages with file changes that are not relevant to logging statements, or added files outside the target programming languages. To address this, we created diff files by comparing the state of files before and after each commit. For JS, we further excluded library and minified files, applying ChromeDevTools (2025)’s minified file-detection algorithm. We then examined the *before* versions of files to locate original logging statements (e.g., *console.log* or *System.out.println*). When such statements appeared at line numbers corresponding to deleted lines in the diff, we verified this as an ad-hoc log and recorded them as data points, capturing the statement text along with contextual information (e.g., file path, enclosing function, line number, commit sha, and function complexity metrics) in a separate CSV file.

Contextualization: Logging statements alone provide limited insight, so we enriched them with predefined metadata for static analysis. This metadata includes the functions and blocks containing each log, visibility of the functions, annotations, parameter types of log statements, and the Cyclomatic Complexity metric, as discussed in Ebert et al. (2016), of each enclosed function. The analysis scripts and metadata are available at Chou (2026b). To detect removed code lines, we inspected diff files for deleted logging statements. Later, we parsed the abstract syntax tree of each targeted source file,

Table 1 Comparison of deleted logging/print statements across languages. *S* denotes the dataset collected with the stricter regex, which requires commit messages to explicitly indicate accidental or debugging intent. *L* denotes the dataset collected with the looser regex, which only requires commit messages to indicate the removal of logging or print statements.

Language	# Commits /w Logs	# <i>L</i> Logs	# <i>S</i> Logs
JavaScript	215,160	548,880	15,090
Python	74,392	220,348	27,049
Java	30,093	113,210	16,975

traversing from each node to identify its enclosing blocks and functions. As JavaScript allows developers to define anonymous functions, we inferred names for such a function as illustrated in Figure 2. Our scripts also support analysis of both major Python versions: 3.X (190,695 print statements) and 2.X (29,653 print statements).

Validation: Because our dataset is derived from simply removed logging statements, there is the potential risk that some collected cases may be core-functionality-based logging — not specifically ad-hoc logs used for runtime comprehension. For example, developers may remove print statements that were originally intended for user-facing output or other purposes. To account for this possibility, we sought to assess the reliability of two options for log curation: one option that is this looser removal-only scheme, and one option that is stricter, which requires more explicit knowledge of developer intention of some debugging or comprehension activity. We anticipate a precision–coverage tradeoff between the two: the looser scheme might admit more false positives, whereas the stricter scheme might exclude valid ad-hoc logs. Specifically, the looser matching scheme identifies commit messages that contain both a removal-related verb and a noun referring to a common debugging statement, such as `console.log` or `print`. The stricter one requires a removal-related verb, an adjective indicating debugging or accidental intent, and a noun describing the removed statement. This stricter pattern is intended to capture commit messages such as “remove accidental print statements” or “delete debugging console.log.” The complete regular expressions are provided in the replication package.

For each language and filtering scheme, we randomly sampled 100 commits from the commits containing logs collected by that scheme and examined every removed print or logging statement in the selected commits. The codebook, provided in the replication package, classified a statement as ad-hoc when its change and surrounding code indicated temporary debugging or runtime comprehension use. This included lightweight checkpoint-style messages (e.g., `print("here")`) and direct printing of variables. Statements serving application behavior, such as output shown to a user through a command-line or user interface, were not classified as ad-hoc logs. The manually annotated sampled commits and codebook are available at Chou (2026a).

The first two authors independently labeled the samples. They then held a four-hour consensus meeting in which they reviewed all sampled statements,

```

Variable Declarator : var cb = function () => {...}
Call Expression : fetch(...).then() => {...}
Assignment Expression : cb = function() => {...}
Class Property : class Text {getText = () => {...} }

```

Fig. 2 Custom approach to assign names to anonymous functions in JavaScript. Tokens highlighted in red represent the names assigned to these functions. We use such method to conduct analysis on function names later.

discussed disagreements against the codebook, and assigned the final labels reported here.

In the looser dataset, manual labeling showed that most sampled logs were indeed ad-hoc: 264 of 269 in JavaScript (98.14%), 276 of 319 in Java (86.52%), and 148 of 183 in Python (80.87%). In the stricter dataset, the proportions were even higher: 156 of 156 in JavaScript (100%), 218 of 235 in Java (92.77%), and 161 of 168 in Python (95.83%). However, Table 1 also shows a great difference in dataset size between the two schemes. The looser scheme collected 548,880 JavaScript logs, whereas the stricter scheme retained 15,090.

Although we sampled 100 commits for each language and filtering scheme, we labeled every removed print or logging statement within those commits. Therefore, the reported validation results are statement-level proportions. Representative false positives included print statements replaced by application or third-party logging facilities and user-facing output removed during broader logic changes.

Because validation began with commits already selected by the regular expressions, it estimates precision among matched commits but cannot estimate how many valid ad-hoc-log removals occurred in unmatched commits. We therefore describe the reduction from the loose to the strict dataset as a coverage tradeoff rather than as a measured recall value. Even the stricter filter should not be interpreted as guaranteeing perfect precision, since developers may use inaccurate or recycled commit messages that do not faithfully describe the actual change. We report results from datasets in the following sections, denoting the looser dataset as *L* and the stricter dataset as *S*.

These results also suggest that JavaScript’s `console.log` is a particularly reliable indicator of ad-hoc logging, whereas `print` statements in Java and Python serve a broader range of purposes, including user-facing command-line output, making them harder to distinguish from non-ad-hoc usage. Despite this challenge in Java and Python, our above validation found 80–88% true ad-hoc logs in the looser set and 92–96% in the stricter set, which should provide an accurate-enough dataset to allow us to derive insights at scale.

5.2 Qualitative Analysis of Live Streaming Data

To investigate how developers employ ad-hoc logs for addressing RQ3 and RQ4, we analyzed live-streaming data by first locating debugging episodes

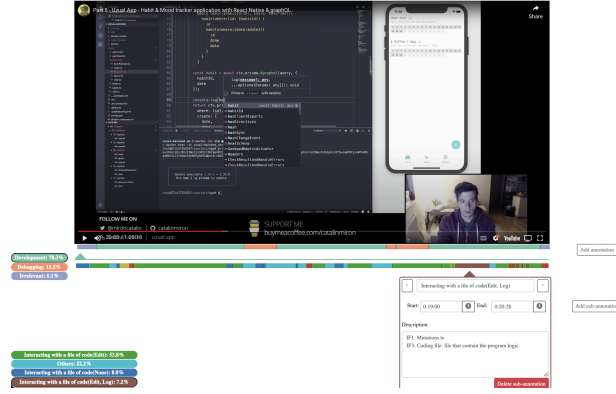


Fig. 3 Online YouTube Videos with the Annotation Platform Observe.dev

within the labeled dataset created by Alaboudi and LaToza (2023). We focused on instances marked as “interacting with a file of code (Edit, Log)” or “interacting with a file of code (Log).” For each segment, we replayed the recording—rewinding or fast-forwarding as needed—until we could respond to a set of predefined guiding questions used in our deductive thematic analysis as suggested in Braun and Clarke (2012).

Figure 3 illustrates a screenshot from Observe.dev, capturing a developer inserting a log into their codebase. We used the following guiding questions to examine streamers’ coding actions and verbal explanations:

- How and why did they insert the log statement?
- When and how did they remove the log statements?
- Where and what exactly did they log?

The initial qualitative coding phase began with the first author reviewing five videos, identifying 45 ad-hoc logs, and developing an initial codebook, which is provided in the replication package at Chou (2026b). The second author then independently coded the same set of videos using this codebook. Inter-rater reliability between the two coders was high (Cohen’s $\kappa = 0.849$). Any disagreements or uncertainties were subsequently resolved through team discussions during weekly meetings until consensus was reached. After this calibration process, first author then proceeded with coding the remaining dataset. In total, fifteen videos from Alaboudi and LaToza (2023), amounting to more than 36 hours of material, were analyzed, resulting in the identification of 92 ad-hoc logs. Videos were labeled sequentially from V1 to V15. Three videos contained no ad-hoc logs: V5 and V6 (Rust) relied solely on compile-time error logs, while V14 (CSharp) consistently used the Visual Studio debugger.

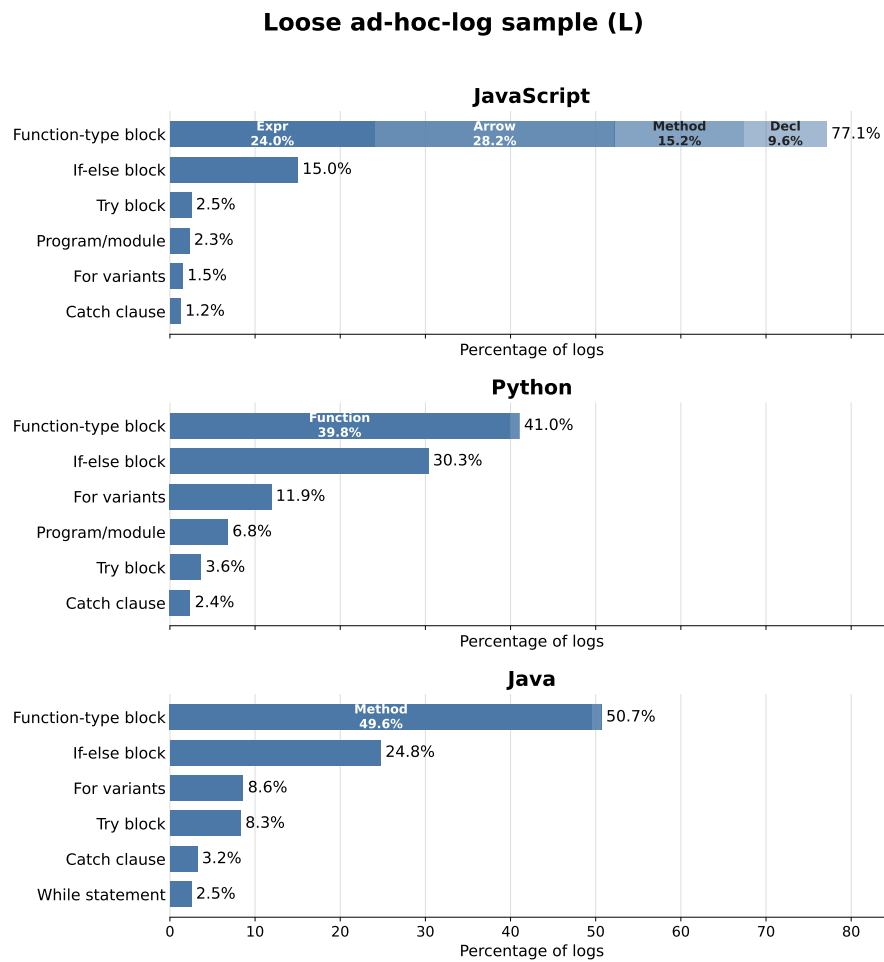


Fig. 4 The six most frequent enclosing block types in the looser ad-hoc-log sample (L), shown separately for JavaScript, Python, and Java.

6 Results

Using the methodology described in the prior two sections, we produced a number of results and findings, which we present here, organized by the research questions that we seek to answer.

6.1 RQ1: Where in the codebase do developers place ad-hoc logs?

Figures 4 and 5 show where developers placed ad-hoc logs across JavaScript, Java, and Python. The largest share appears in **function-type blocks** in all three languages in both stricter and looser datasets: **77.1% (L)** and **74.4%**

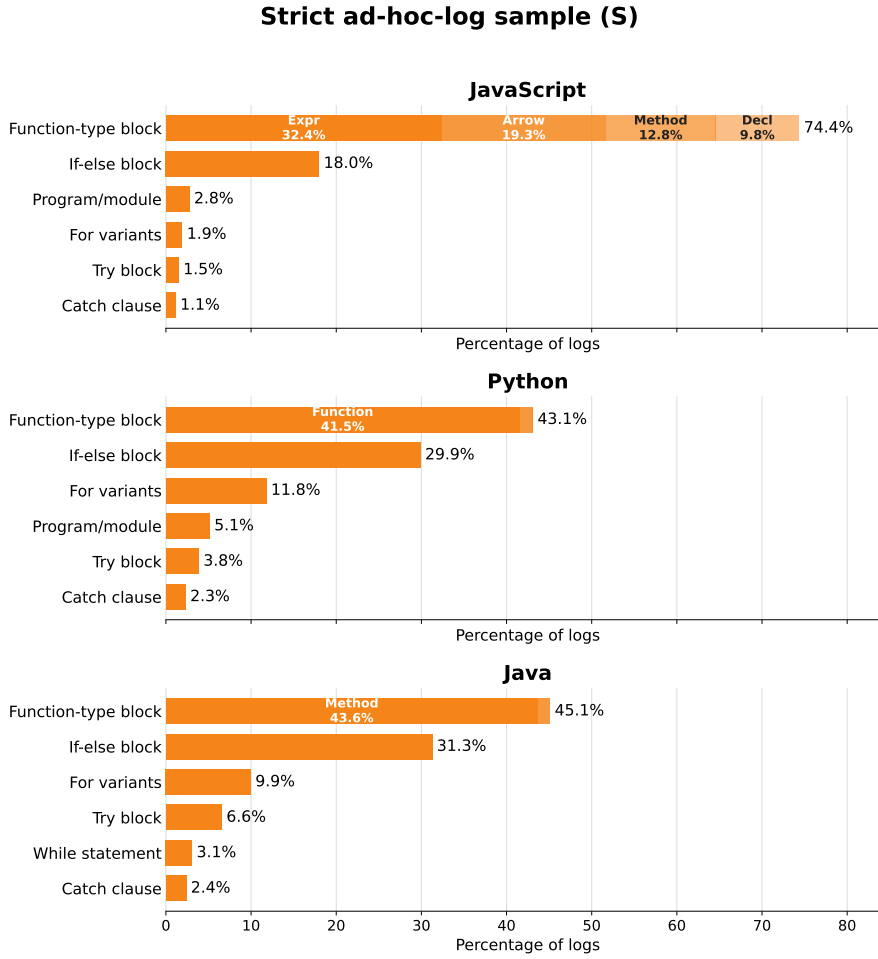


Fig. 5 The six most frequent enclosing block types in the stricter ad-hoc-log sample (S), shown separately for JavaScript, Python, and Java.

(S) in JavaScript, **50.7% (L) and 45.1% (S)** in Java, and **41.0% (L) and 43.1% (S)** in Python. By “function-type block,” we mean that the log statement resides directly in the body of a function or method, rather than nested within a conditional, loop, or error-handling block. This distinction separates, for example, a `console.log` at the top of a function from one placed inside an `if` branch or a `try` block. Because the language datasets differ in collection window, event sampling, extraction and filtering, and validation precision (Sections 4 and 5.1), their absolute percentages should not be compared as estimates of language effects. We use them to characterize placement patterns within each language. If-else placements are the next largest group: **15.0% (L) and 18.0% (S)** in JavaScript, **24.8% (L) and 31.3% (S)** in Java, and **30.3% (L) and 29.9% (S)** in Python. Error-handling blocks are a mi-

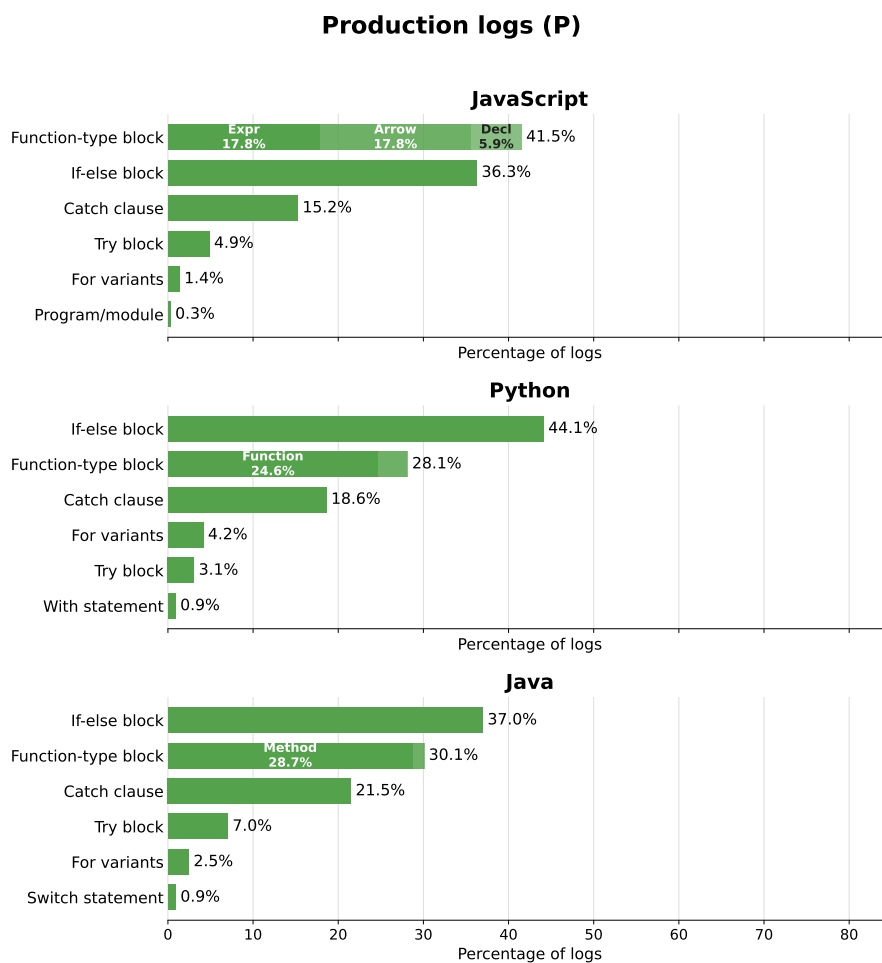


Fig. 6 The six most frequent enclosing block types among production logs, shown separately for JavaScript, Python, and Java.

nority in the ad-hoc-log samples. For example, JavaScript `TryStatement` and `CatchClause` placements together account for less than 4% in both L and S.

These patterns differ from Li et al. (2020)’s findings on *production* logs, which showed that logging statements are commonly placed in error-handling and conditional blocks. Li et al. (2020) focused on industrial Java systems, whereas our dataset is primarily drawn from open-source projects across multiple languages. To support a like-for-like comparison, we also collected production logs from the same repositories using the procedure described in §4 and examined their surrounding block types, as shown in Figure 6. Our production-log sample is broadly consistent with prior work: production logs are more concentrated in `catch/except`-like blocks and `if-else` branches. For example, in our JavaScript production-log sample, `CatchClause` accounts for 15.25%

of logging statements and `IfStatement` accounts for 36.28%, compared with 1.2% and 15.0%, respectively, in the JavaScript L sample. Java shows the same pattern: production logs are concentrated in `CatchClause` (21.49%) and `IfStmt` (36.98%), with an additional 7.02% appearing in `TryStmt`. Ad-hoc logs are less frequent in these locations and are more often placed directly in function or method bodies.

This contrast is visible across languages. In JavaScript, function-related blocks remain prominent even for production logs: `FunctionExpression`, `ArrowFunctionExpression`, `MethodDefinition`, and `FunctionDeclaration` together account for 41.51% of the sample. Across all three languages, production logs are more concentrated in exception-handling and conditional blocks, whereas ad-hoc logs are more concentrated at the immediate function- or method-body level. Taken together, this pattern suggests that ad-hoc logs often serve as short-term aids for runtime comprehension during active development, whereas production logs more often record branch outcomes and error states in deployed systems.

Finding 1A: Ad-hoc logs were placed primarily at the top level of function blocks (e.g., 77.1% in JS, 41.0% in PY in L set), in contrast to production logs, where about half reside in catch handlers or branch blocks as discussed in Li et al. (2020); Fu et al. (2014); Yuan et al. (2012).

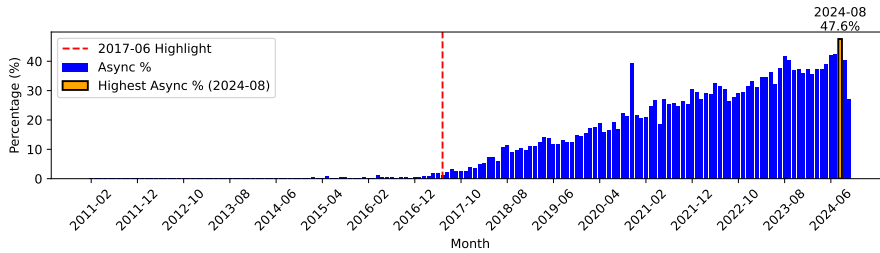
Types of Functions with Ad-hoc Logs. Understanding JavaScript systems is often difficult for developers because of the language’s heavy reliance on asynchronous and event-driven functions. As such, this complexity has motivated significant research on visualization and static analysis techniques aimed at improving debugging for such features (e.g., Alimadadi et al. (2016); Turcotte et al. (2022)).

In this work, we investigate whether ad-hoc logs are more commonly used in certain function types—specifically anonymous, callback, and asynchronous functions—to aid developers’ runtime understanding. To establish a baseline, we applied the sampling method described in Section 5.1 and report the distributions of function types across three samples in Table 2. Results show that anonymous functions dominate overall, while asynchronous functions account for less than 10%, and callbacks comprise fewer than 40%. Async functions are defined with the *async* keyword and typically manage asynchronous tasks such as HTTP requests. We label functions declared inline at a call site as “callback functions.” Anonymous functions, as expected, are those without names.

We also examined how these distributions changed over time. Figure 7 shows only the trend for asynchronous functions in sample *L*, since the proportions for callback and anonymous functions remain largely stable; the full results for those categories are included in the replication package at Chou (2026b). We report only the trend for sample *L* for two reasons. First, manual inspection showed that the JavaScript data has high accuracy in both the looser and stricter samples. Second, the stricter sample contains too few commits and removed logs to reveal a clear temporal trend. Notably, the share

Table 2 Function-type distributions in the three JavaScript baseline samples. Cells report count (percentage). Categories can overlap, so row percentages do not sum to 100%.

Sample	Function type	Total #functions	Count	Percentage
2023–2024	Async	1,070,838	46,940	4.38%
	Anonymous		664,649	62.07%
	Callback		249,743	23.32%
	Other		377,939	35.29%
GitHub Archive	Async	2,937,684	18,628	0.63%
	Anonymous		2,393,852	81.49%
	Callback		902,542	30.72%
	Other		516,581	17.58%
Google BigQuery	Async	2,009,471	146,351	7.28%
	Anonymous		1,725,954	85.89%
	Callback		771,356	38.39%
	Other		262,625	13.06%

**Fig. 7** Percentage of ad-hoc logs within asynchronous functions over time in sample *L*

of ad-hoc logs appearing in asynchronous functions has risen steadily since the introduction of the *async* keyword in mid-2017, reaching 47.6% by August 2024. This figure contrasts sharply with our 2023–2024 repository sample, in which only 4% of all collected functions are asynchronous. Similarly, our qualitative analysis of JavaScript tutorial videos shows that 45.3% of ad-hoc logs (24 out of 53) appear in *async* or *callback* functions.

Prior literature, such as Alimadadi et al. (2016); Trümper et al. (2010), suggests that multithreading and concurrent programming are particularly challenging for developers to understand. To examine this in our context, we compared classes containing removed ad-hoc logs with a baseline drawn from 2,000 Java repositories.

For each Java source file, the analyzer examined the first top-level type declared in that file. We classified that type as a “concurrency-type class” when its declaration directly extended a recognized concurrency-related class (e.g., `Thread`) or directly implemented a recognized concurrency-related interface (e.g., `Runnable`). The detector checks the type names written in the `extends` and `implements` clauses; it does not infer indirect inheritance. In the baseline, 3,165 of 760,399 classes (0.42%) matched the indicator. Among classes containing at least one removed ad-hoc log statement, 975 of 44,166 class observations (2.21%) matched it, corresponding to a risk ratio of 5.30.

Table 3 Top-10 method/function names with ad-hoc logs across Java, JavaScript (functions), JavaScript (callbacks), and Python.

Rank	Java	JS (Functions)	JS (Callbacks)	PY
1	test_0	render	then	main
2	main	constructor	on	__init__
3	run	componentDidMount	for each	run
4	execute	success	catch	forward
5	test_alter_first	handleSubmit	subscribe	train
6	actionPerformed	exports	get	get
7	update	init	post	update
8	handle	ngOnInit	map	post
9	doGet	update	it	__call__
10	initialize	onSubmit	add event listener	_lint_html_files

Thus, concurrency-type classes were more frequent in the log-removal sample, although they remained a small minority in both groups. These percentages describe the direct source-code relationships captured by the indicator, not all forms of Java concurrency.

Name of the functions with ad-hoc logs. Across languages, we observed from two perspectives: (i) the names of functions or methods themselves where ad-hoc logs are present, and (ii) in JavaScript specifically, the names of functions that *accept* callbacks containing ad-hoc logs. When considering functions and methods directly, we find that each ecosystem reflects its dominant paradigm. In JavaScript, ad-hoc logs are concentrated in lifecycle methods such as *render*, *constructor*, *componentDidMount*, and *ngOnInit*, pointing to the prevalence of logging in frontend component initialization and update flows. In Java, the most frequent names include *main*, *run*, and *execute*, which correspond to entry points and execution control, but also testing and GUI-related names such as *test_0*, *test_alter_first*, and *actionPerformed*. In Python, the top names are more generic, reflecting its use across diverse domains: common lifecycle functions (*__init__*, *__call__*), general execution (*main*, *run*, *update*), and some workload-specific routines, for example in machine learning (*forward*, *train*) and developer tooling (*_lint_html_files*). Unlike Java and JavaScript, no single pattern dominates beyond the natural diversity of Python applications. Meanwhile, in JavaScript’s callback-accepting functions, the most common names are *then*, *catch*, and *on*, highlighting asynchronous and event-driven control. Taken together, these distributions suggest that ad-hoc logs appear where developers most often need to reason about complex, framework-controlled behaviors: lifecycle timing in JavaScript UIs, concurrency and control flow in Java, and a wide range of tasks shaped by Python’s flexibility across domains.

Finding 1B: In JavaScript, many ad-hoc logs occur in callbacks, asynchronous functions, and library lifecycle functions. The proportion occurring in asynchronous functions increased from the standardization of `async/await` in 2017 through 2024. In Java, concurrency-related classes were more frequent among classes containing removed ad-hoc logs than in the baseline (2.21% versus 0.42%). Together, these descriptive results suggest that developers use ad-hoc logs in contexts where indirect or framework-controlled runtime behavior may be difficult to observe.

Because the complexity distributions are skewed, we compared the groups using two-sided Mann–Whitney U tests and report Vargha–Delaney A as the effect size. The effect size is the probability that a randomly selected observation from the first group has greater complexity than one from the comparison group, with ties receiving half weight; $A = 0.5$ indicates no distributional tendency.

Table 4 Cyclomatic complexity of all functions and functions containing removed ad-hoc logs. The Mann–Whitney p -values and Vargha–Delaney A effect sizes compare each ad-hoc group with all functions.

Dataset	Group	Mean	Median	Std.	p vs. All	A vs. All
BigQuery (JS)	All	3.32	2.0	6.20	–	–
	Ad-hoc (L)	7.77	4.0	19.19	< 0.001	0.699
	Ad-hoc (S)	7.80	4.0	11.96	< 0.001	0.711
GitHub (JS)	All	3.64	2.0	155.06	–	–
	Ad-hoc (L)	5.62	3.0	10.21	< 0.001	0.616
	Ad-hoc (S)	6.91	3.0	14.37	< 0.001	0.652
Java	All	1.88	1.0	3.43	–	–
	Ad-hoc (L)	6.55	3.0	12.51	< 0.001	0.762
	Ad-hoc (S)	7.86	4.0	14.16	< 0.001	0.824
Python	All	2.56	1.0	4.14	–	–
	Ad-hoc (L)	6.99	3.0	23.56	< 0.001	0.597
	Ad-hoc (S)	7.32	4.0	12.41	< 0.001	0.691

Table 5 Cyclomatic complexity of functions containing production logs and rank-based comparisons with the ad-hoc-log datasets. Vargha–Delaney A is oriented as production versus ad-hoc.

Language	Mean	Median	Std.	p_L	$A_{P,L}$	p_S	$A_{P,S}$
JavaScript	8.59	5.00	13.69	< 0.001	0.590	< 0.001	0.577
Java	9.09	4.00	29.18	< 0.001	0.572	0.007	0.506
Python	11.38	6.00	36.79	< 0.001	0.666	< 0.001	0.619

We examined whether functions containing removed ad-hoc logs differed in observed cyclomatic complexity from the sampled function populations. Comparing cyclomatic complexity across datasets, we found that logged functions

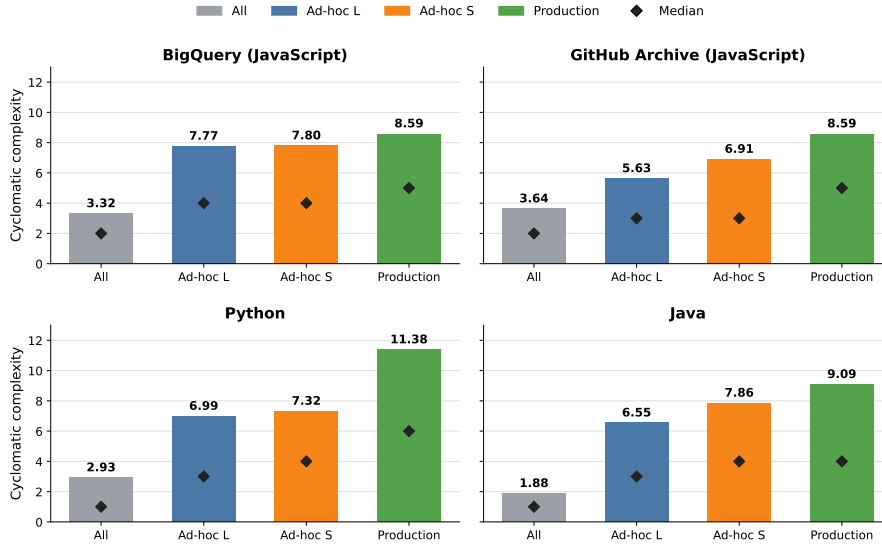


Fig. 8 Cyclomatic complexity of all functions, functions with removed ad-hoc logs, and functions with production logs across datasets. Bars show mean complexity, while diamond markers indicate medians.

are consistently more complex. In JavaScript (BigQuery and GitHub Archive), logged functions had higher mean and median complexity than all functions (BigQuery: $\mu = 7.77$, $\tilde{x} = 4.0$ vs. $\mu = 3.32$, $\tilde{x} = 2.0$; GitHub: $\mu = 5.62$, $\tilde{x} = 3.0$ vs. $\mu = 3.64$, $\tilde{x} = 2.0$), with $p < 0.001$ in both cases. The same pattern appeared in Java (6.55 vs. 1.88, $p < 0.001$) and Python (6.99 vs. 2.56, $p < 0.001$). Taken together, functions containing removed ad-hoc logs had higher observed cyclomatic complexity across the studied datasets, as summarized in Table 4 and Figure 8.

Functions containing production logs also had higher mean complexity than functions containing removed ad-hoc logs in all three languages (JavaScript: 8.59; Java: 9.09; Python: 11.38). The rank-based effect sizes, however, show that the magnitude varies across comparisons. For production logs versus L , Vargha–Delaney A was 0.590 in JavaScript, 0.572 in Java, and 0.666 in Python. Against S , it was 0.577, 0.506, and 0.619, respectively. Thus, the Java production-versus- S distributions were nearly indistinguishable despite the difference in their means. These results are summarized in Table 4, Table 5, and Figure 8. These comparisons describe unadjusted associations; function size, project characteristics, and other structural properties may influence both cyclomatic complexity and logging activity.

Finding 1C: Across the studied datasets, functions containing removed ad-hoc logs had higher observed cyclomatic complexity than the sampled overall function populations.

Table 6 Top-10 Most Frequent Literal Argument Strings Across Languages

Rank	JavaScript	Python	Java
1	NUMBER	-R	-R
2	here	\n	Transaction started .R
3	-R	=R	NUMBER
4	test	NUMBER	Transaction committed .R
5	hello	)	*R
6	hi	here	=R
7	success	#R	\n
8	error	*R	Starting standalone server.R
9	=R	,	testRead() .R\n
10	clicked	+R	Transaction interrupted .R

6.2 RQ2: What information do developers record in ad-hoc logs?

Ad-hoc Logs with Variable Labels. Most logs contain only a single variable or literal (72.6% in JS, 80.5% in PY, 95.1% in JAVA) or two arguments (24.2% in JS, 14.0% in PY, 0.6% in JAVA). Consistent with prior findings (Jiang et al. (2023)), developers often add labels to help trace logged variables back to their origins in the console output. Our analysis shows that when a log includes two arguments, 84.7% in JS and 76.8% in PY contain at least one literal. (We excluded Java from this analysis since only 0.6% of print statements have two arguments.) Among these, 51.5% (JS) and 38.8% (PY) explicitly reference the other argument within the literal. For example, in `console.log('Results: ', results)`, the string label “Results: ” clarifies the meaning of the logged variable value, making it easier to locate in the output stream.

Ad-hoc Logs Formats. Table 6 presents the top 10 most frequent literal contents in ad-hoc logs for each programming language. The symbol *R* denotes a sequence of characters repeated more than three times; for example, `=R` represents repeated equal signs such as “====,” often used to create separation markers or to highlight certain content in the log output. The most dominant entries across the three languages are template-like strings, which often serve as lightweight markers in logs. Strings such as *here*, *clicked*, and *hi* are frequently used to indicate control-flow checkpoints or event triggers during execution. In the case of Java, transaction-related lifecycle markers (e.g., *Transaction started*, *Transaction committed*) are prevalent, reinforcing our observation in Finding 1B.

In live-stream video coding, the percentages are similar for the log parameter count, where the logs contain one (69.6%) or two (19.6%) arguments, and 15 out of 18 (83.3%) two-argument logs are structured as a label string and variables. Furthermore, 9 out of 15 (77.8%) contain the name of the variable. In addition, similar to the `JSON.stringify` function, developers using C/C++ tend to create formatting functions of their own. In V4, V7, V12, the developers wrote their own debug function `infof`, `dbg`, `dbgprintf` to simplify forming a more readable log output. When logging variables without native serialization methods, the developers in V4 and V7 wrote helper functions to output variables in a readable string.

Finding 2: To locate variables and to understand execution order, developers structure the output of ad-hoc logs through labeling and formatting strings akin to findings from prior work (Jiang et al. (2023)).

6.3 RQ3: Why do developers rely on ad-hoc logs?

To answer RQ3 and RQ4, we shifted our focus to live-streamed data to address more qualitative questions and gain insight into why and how developers use ad-hoc logs. From analyzing and coding 15 live-stream videos and 92 obtained ad-hoc logs, we summarize our key findings as follows:

Understanding Program States (n=68): Developers often use ad-hoc logs to observe or compare program states for several reasons. A common reason is to capture states that are not immediately clear from the source code, such as states introduced by a third-party library or fetched via a web API. For example, the developer in V3 expressed uncertainty about a library function call’s effect and logged a boolean value triggered by a GUI click event, noting, *“Does it return true when option changes, or does it return true when it set it on? That’s the thing I don’t know. Let’s find it out.”* Developers also log multiple states to cross-reference their values, ensuring they align with expectations. For instance, the developer in V13 checked three boolean values with logs to iteratively verify they matched expectations. In other cases, logs are used to check the state of a single variable or condition, providing quick feedback on specific program values. To understand the initial program states when debugging the function, developers also inserted the logs right after the function to inspect the parameters. For example, the developer in V9 logged the parameter right after the function to check how the function works, *“al-right, let’s see what this does.”* Developers also crafted their own formatting function when dealing with complex or non-readable program states. For example, the developer in V4 working on the curl project introduced a function `what2name` to transform a constant integer to a string, which increased the readability of logging outputs.

Understanding Execution Flow (n=33): Logging is often used to understand or verify code execution, particularly for those in asynchronous functions or executions from third-party libraries. For example, V13 inserted an ad-hoc log with “----” in a lifecycle method, noting, *“It gives me a spot to look at... yep, looks like the componentDidUpdate did not get called when I set the props.”* Another reason is sanity checking (n=31), where developers insert strings or variables to confirm that the program reaches specific points. For example, V1 logged “IM GOING TO RUN HERE” to verify if the software actually reaches to that specific point.

Finding 3: Developers added ad-hoc logs to gain insight into program states and execution flow, particularly in cases that were not immediately obvious—such as asynchronous functions or behaviors introduced by third-party libraries. These logs were also used to validate assumptions about expected runtime behavior and to carry out sanity checks.

6.4 **RQ4:** How do developers manipulate ad-hoc logs (e.g., insert, revise, or remove) to achieve their goals?

Our study found that developers employed multiple approaches to create and refine ad-hoc logs, such as iterative modification, comparative checks, varied timing, and strategies for removing logs. The main themes derived are:

Iterations (n=27): Developers progressively refined their logs to better understand program states and identify optimal logging points or variables. For instance, V15 revised the same log three times, evolving from *console.log(this.props)* to *console.log(this.props.navigation.action)* across multiple edit–run cycles. Likewise, V7 made six adjustments while tracing the call stack of a double-click event, moving the log back one level each time, and explained, “*I don’t remember how all this works, so we are just gonna print to make sense of it.*”

Comparison (n=22): Developers leveraged logs to contrast program states, either by grouping several states within one log statement or by distributing them across multiple logs. They often added labels (e.g., variable names) to link outputs back to the source code. For instance, V4 recorded the same variable at two execution points with distinct labels to verify whether the values matched.

Developers also inserted ad-hoc logs at multiple points in the code to compare the sequence of execution, especially within complex logic or asynchronous flows. For example, V2 placed two dummy strings in different locations and tracked their order of appearance in the log output as various events unfolded.

Varied Timing of Ad-hoc Log Removal: As anticipated, most ad-hoc logs were discarded once they fulfilled their immediate purpose. Specifically, many logs were deleted right after their output was inspected (n=61), and most were observed or confirmed to have been eventually removed (n=80). For the remaining cases, some logs could not be confirmed because the recordings ended before their fate was observed (n=7), while others were ultimately committed to the repository (n=5). Among the committed logs, three were intentionally retained as they evolved into production code (n=3), and two appeared to be accidental commits (n=2). Noteworthy instances include V1 storing logs in a *git stash* for potential reuse during future debugging, and V13 being prompted to remove a log immediately after viewing its output in the terminal.

Varied Strategies for Ad-hoc Log Removal: The strategies employed for ad-hoc log removal were diverse. While direct deletion was the predominant approach ($n=54$), several alternative practices were observed. For example, V1 avoided committing files containing logs ($n=2$), and V9 temporarily commented out one log to facilitate run-to-run comparisons ($n=2$). In V13’s case, a linter tool prompted the removal of ad-hoc logs ($n=1$), and the developer also created and used an external environment (CodeSandbox) to capture debugging outputs in real time without altering the main codebase ($n=3$). However, not all log management actions were deliberate—V15 inadvertently committed two ad-hoc logs after forgetting to delete them ($n=2$).

Finding 4: We identified two primary strategies in the way developers authored ad-hoc logs: (1) iteratively refining the content or placement of logs, and (2) inserting multiple logs at once to capture information in parallel. The differing timings and varied approaches to removing these logs indicate that their deletion is often as ad-hoc as their creation.

7 Discussion

Perils and Promises of Learning from Mistakes. As Henry Lieberman noted in (Lieberman (1997)), “debugging is the dirty little secret of computer science.” This “dirt” has only accumulated over time, with using debuggers to understand runtime behavior often being regarded as the default solution, leading some developers to feel ambivalent or even reluctant to admit their use of “printf debugging” techniques (Beller et al. (2018)). Such hesitation limits the research community’s insight into debugging behaviors, like ad-hoc print statement usage, which play a significant role in shaping final patches or feature development. Our research uncovers this “dirt,” through mining developers’ mistakes to offer the community new possibilities to understand ad-hoc logging practices.

Alaboudi and LaToza (2021) conducted analysis of live-streaming data and highlighted that software development is an iterative process involving frequent edit-run cycles. However, these small cycles are often hidden within local development environments, limiting the feasibility of large-scale studies. Previous research has addressed this challenge by either conducting costly lab studies (Gilmore (1991)), analyzing live-streaming videos (Alaboudi and LaToza (2019)), or requiring developers to install intrusive IDE plugins (Aniche et al. (2022); Beller et al. (2018)). Our method opens new possibilities for scalable insights into these cycles.

While our collection of ad-hoc logs from revision-control systems enables large-scale analysis, there are limitations: (1) the mined data provides only a snapshot of logs unintentionally committed to version control, making it difficult to observe iterations in the log-authoring process; (2) although the surrounding source code offers some context, it does not capture the developers’ rationale for adding, removing, or evaluating the utility of these logs. To

gain deeper insight into the context, reasoning, creation, and deletion of these logs, we mined and analyzed the live-streaming data.

Moreover, the distribution of the repository reveals that our dataset, similar to the repositories reported in Kalliamvakou et al. (2014), is skewed toward smaller, personal, and often inactive repositories. We chose not to explicitly exclude personal projects, as this distribution presents a double-edged sword for our analysis. Whereas large, well-maintained open-source projects often enforce linter checks on pull requests to prevent ad-hoc logs, as suggested by Foundation (2025), personal projects—wherein maintainers might carelessly push ad-hoc logs—allowed us to perform large-scale analysis. Future researchers using our dataset or similar methods should proceed with caution, as data without fine-grained filtering may not fully generalize to expert developers or contributors in collaborative open-source projects.

Markers in the Forest of Complex Software Execution. Each logging statement placed by developers can be understood as a marker left within the forest of execution, intended to make sense of code that would otherwise be difficult to navigate. Our analysis shows that ad-hoc logs are frequently inserted into threading-related code, asynchronous functions, callbacks, and the lifecycle methods of third-party libraries (Finding 1B). These contexts complicate reasoning because the order and timing of execution are often outside the developer’s immediate control. Logs in such locations serve as reference points for reconstructing runtime behavior. In addition, functions containing removed ad-hoc logs had higher observed cyclomatic complexity than the sampled function populations across all three languages (Finding 1C). This association is consistent with developers using logs in code that is structurally difficult to reason about, although the similar pattern among production logs indicates that it is not unique to ad-hoc logging.

While these findings may appear intuitive, the patterns of where developers place logs provide an opportunity to re-examine how software complexity is measured. The debate over which metrics—Cyclomatic Complexity, Cognitive Complexity, or custom alternatives—best capture developer-perceived difficulty has persisted for decades, such as discussion in Shepperd (1988); Scalabrino et al. (2017). Since controlled human studies remain expensive and difficult to scale, our dataset offers a complementary empirical lens and motivates future work examining whether logging behavior can complement established measures of code complexity and developer-perceived difficulty. This opens the door for future work to validate or refine existing metrics, or to propose new ones grounded in observed developer practices. In applied contexts, educators and maintainers could also use patterns of ad-hoc logging as a proxy for identifying code that is unusually challenging, helping to target refactoring efforts or calibrate the level of instructional material. For instance, the high frequency of logs used to understand React component lifecycles in JavaScript may point to aspects of the API design that warrant reconsideration or a separate visualization to support comprehension.

A single marker is rarely enough to navigate complex execution paths. As shown in Finding 4, developers often placed logs iteratively, refining their placement until the information was sufficient to understand program states and flows. This iterative pattern indicates that initial logs are frequently sub-optimal for comprehension, functioning more as tentative probes than definitive solutions. These observations highlight an opportunity for future research: by building on insights from production log studies, it may be possible to recommend more effective initial logging locations such as techniques introduced in Fu et al. (2014); Li et al. (2020) and variables like Liu et al. (2021). Such guidance could reduce the trial-and-error effort developers currently expend, allowing them to converge on useful runtime insights more quickly.

Effective logging is not only about where statements are placed, but also about the richness of the information they convey. Finding 2 shows that many ad-hoc logs extend beyond a simple message: developers often include labels, formatting strings, or custom debug helpers. Finding 3 further shows that developers craft anchor strings and selectively structure complex objects so that logs can be compared across states and used to trace execution over time. Together, these findings suggest that developers treat ad-hoc logs as deliberate reasoning aids rather than arbitrary print statements. This observation has direct implications for tool design. IDE support should not only recommend where to insert logs, but also help developers generate informative logs by suggesting discriminative anchor strings, relevant variable groupings, and readable object summaries tailored to the local context. Such support could be especially valuable in asynchronous or callback-heavy code, where developers must mentally map console output back to source locations and execution order. Our findings also suggest practical logging guidelines: developers benefit from logs that include stable textual anchors, explicit state summaries, and enough contextual detail to distinguish nearby execution points. Existing tools such as Log-it Jiang et al. (2023) begin to reduce the burden of connecting output to code, while prior work on visualization Turcotte et al. (2022) and logging recommendation Li (2020) points to promising directions for operationalizing these insights in future developer-support tools.

Exploratory recursion sensitivity analysis. We also considered recursion as a potential source of comprehension difficulty, since prior work suggests that recursive structures and code constructs may introduce cognitive complexity not fully captured by standard cyclomatic complexity as discussed by Hao et al. (2023); Ajami et al. (2019). As a post hoc analysis, we examined whether direct recursion might explain the complexity pattern in Finding 1C. This analysis is exploratory and is not part of the confirmatory RQ1 findings. We applied a source-code self-call indicator consistently to the baseline and ad-hoc populations within Python and Java.

In Python, the indicator marks a function when its body contains a direct call written as `f()` or `self.f()`, where `f` is the enclosing function’s name. It identified 792 of 74,219 functions containing removed ad-hoc logs (1.067%) and 20,268 of 2,293,489 baseline functions (0.884%), a prevalence ratio of 1.208

(95% CI [1.125, 1.296]). In the retained subset used for the exclusion sensitivity analysis, removing functions marked as recursive changed the estimated difference in the geometric mean of one plus cyclomatic complexity only slightly, from 32.65% to 32.45%.

In Java, the indicator marks a method when its body contains a direct call written as `f()`, `this.f()`, or `C.f()`, where `f` is the enclosing method's name and `C` is the enclosing class's name. It identified 1,110 of 60,932 methods containing removed ad-hoc logs (1.822%) and 316,235 of 12,546,753 baseline methods (2.520%), corresponding to a prevalence ratio of 0.723 (95% CI [0.682, 0.766]). After methods marked as recursive were excluded, mean cyclomatic complexity remained higher among methods containing removed ad-hoc logs than among baseline methods (4.82 versus 1.87).

Threats to Validity.

Internal Threats to Validity. For the mined ad-hoc logs, a key internal threat is that some logs classified as ad-hoc may actually be removed production logs, which could affect the accuracy of our findings. To address this, we manually labeled 100 sample commits for each language and found that only a small portion were not used in an ad-hoc manner. In coding the live-streaming data, researcher bias is a common concern due to the subjective nature of qualitative analysis. We mitigated this by involving a second researcher and holding iterative discussions to minimize uncertainty and resolve any disagreements.

Our Java concurrency-type indicator captures the common case in which a method belongs to the primary class in its source file, but it may miss or misattribute methods in nested, anonymous, and secondary types. Because it examines directly declared supertypes, it also does not capture indirect inheritance, executor submissions, lambdas, annotations, or framework and event-dispatch callbacks. The Java comparison therefore concerns this source-code indicator rather than all forms of Java concurrency.

External Threats to Validity. Our findings may not generalize to other programming languages or development contexts. Although our repository analysis covers Java, JavaScript, and Python, the qualitative analysis includes live-stream videos spanning a broader set of languages, and the two datasets should therefore not be interpreted as directly comparable language samples. The live-stream analysis is also based on a relatively small sample and may be subject to performance bias, as developers may behave differently while streaming their work as noted in both Chou et al. (2026) and Alaboudi and LaToza (2019). Moreover, the repositories represented in these videos are skewed toward personal or inactive projects, which may not reflect development practices in actively maintained or industrial projects. While the selection and representativeness of these videos were discussed by Alaboudi and LaToza (2019), our study uses them to investigate different research questions and therefore inherits additional limitations specific to our analysis. These factors limit the generalizability of our qualitative observations, which should be interpreted as exploratory evidence of how developers use ad-hoc logs rather than as representative estimates of broader developer behavior.

8 Conclusion

This paper presented the first large-scale empirical study of ad-hoc, or “printf-style,” logging by systematically mining accidental commits and analyzing live-streamed programming sessions. By leveraging over 800,000 removed log statements across JavaScript, Java, and Python, as well as 36 hours of qualitative observations, we demonstrated that ad-hoc logs are not arbitrary artifacts but purposeful markers developers employ to navigate complex code and runtime behaviors.

Our findings highlight three themes. First, ad-hoc logs are disproportionately concentrated in contexts where reasoning about execution is especially difficult, such as asynchronous and callback functions in JavaScript or thread-related code in Java. Second, both ad-hoc and production logs tend to occur in functions with higher cyclomatic complexity. This shared pattern is consistent with logging serving as a means of observing runtime behavior in structurally complex functions, whether the logs address an immediate debugging need or remain for long-term production use. Third, developers refine ad-hoc logs iteratively, structure them with labels and formatting for clarity, and frequently remove them once their immediate purpose is served—though often with ad-hoc strategies for removal.

Taken together, these results contribute both methodological and practical insights. Methodologically, our approach demonstrates how accidental artifacts in version control can serve as valuable data sources for understanding hidden aspects of software practice. Practically, our results suggest opportunities for tools that recommend effective logging locations, improve log readability, or visualize execution traces, thereby reducing the trial-and-error burden of printf debugging.

Ultimately, ad-hoc logs, though ephemeral, offer a window into developers’ struggles and strategies for program comprehension through aggregation. By learning from these transient traces, researchers and tool builders can better support the everyday practices that shape debugging and development, bridging the gap between lightweight, local strategies and more formalized approaches to runtime understanding.

9 Declarations

9.1 Funding

This research is partially supported by the Dieter Schwarz Stiftung Foundation. Yi-Hung Chou is supported in part by the Taiwan Government Scholarship to Study Abroad.

9.2 Ethical Approval

Not applicable.

9.3 Informed Consent

Not applicable.

9.4 Author Contributions

Yi-Hung Chou led the study, designed the experiments, wrote the manuscript, and developed the crawling and analysis scripts used across all languages. Boyuan Jiang conducted the experiments on Java. Vedit Jain conducted the experiments on Python. Yiyang Min performed the qualitative analysis and supported the JavaScript and TypeScript analyses. April Yi Wang and James A. Jones supervised the research, provided guidance on the experimental design, and contributed to manuscript writing and revision.

9.5 Data Availability Statement

All datasets generated and analyzed during this study are openly available on *figshare*. The manual check of collected logs through 100 commits sample across all programming languages is available at <https://doi.org/10.6084/m9.figshare.27325635.v3>. The replication package, including logs, commits, crawling and analysis scripts, and qualitative analysis on live streams, is available at <https://doi.org/10.6084/m9.figshare.27638058>. The additional dataset and scripts for Java, Python, Stricter JS, as well as the manual validation results can be found at <https://doi.org/10.6084/m9.figshare.33994521>.

9.6 Conflict of Interest

The authors declare that they have no conflict of interest.

9.7 Clinical Trial Number

Clinical trial number: not applicable.

Acknowledgements We extend our gratitude to the anonymous reviewers for their thorough and insightful feedback. We would also like to thank Heng Du, Junling Wang, Xiaotian Su, Zeyu Xiong, Gustavo Umbelino, Behnood Masoudi, Monil Narang for providing feedback and support.

References

Ajami S, Woodbridge Y, Feitelson DG (2019) Syntax, predicates, idioms — what really affects code complexity? *Empirical Software Engineering*

- 24(1):287–328, DOI 10.1007/s10664-018-9628-3, URL <https://doi.org/10.1007/s10664-018-9628-3>
- Alaboudi A (2023) observe-dev.online. URL <https://observe-dev.online>, accessed: 2024-11-09
- Alaboudi A, LaToza T (2023) What constitutes debugging? An exploratory study of debugging episodes. *Empirical Software Engineering* 28(5):117, DOI 10.1007/s10664-023-10352-5, URL <https://doi.org/10.1007/s10664-023-10352-5>
- Alaboudi A, LaToza TD (2019) An Exploratory Study of Live-Streamed Programming. In: 2019 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC), pp 5–13, DOI 10.1109/VLHCC.2019.8818832, URL <https://ieeexplore.ieee.org/document/8818832>
- Alaboudi A, LaToza TD (2021) Edit - Run Behavior in Programming and Debugging. *IEEE Computer Society*, pp 1–10, DOI 10.1109/VL/HCC51201.2021.9576170, URL <https://www.computer.org/csdl/proceedings-article/vl-hcc/2021/09576170/1y63pYboUgg>
- Alimadadi S, Mesbah A, Pattabiraman K (2016) Understanding asynchronous interactions in full-stack JavaScript. In: *Proceedings of the 38th International Conference on Software Engineering*, Association for Computing Machinery, New York, NY, USA, ICSE '16, pp 1169–1180, DOI 10.1145/2884781.2884864, URL <https://doi.org/10.1145/2884781.2884864>
- Aniche M, Treude C, Zaidman A (2022) How Developers Engineer Test Cases: An Observational Study. *IEEE Transactions on Software Engineering* 48(12):4925–4946, DOI 10.1109/TSE.2021.3129889, conference Name: *IEEE Transactions on Software Engineering*
- Beller M, Spruit N, Spinellis D, Zaidman A (2018) On the dichotomy of debugging behavior among programmers. In: *Proceedings of the 40th International Conference on Software Engineering*, Association for Computing Machinery, New York, NY, USA, ICSE '18, pp 572–583, DOI 10.1145/3180155.3180175, URL <https://dl.acm.org/doi/10.1145/3180155.3180175>
- Bird C, Ford D, Zimmermann T, Forsgren N, Kalliamvakou E, Lowdermilk T, Gazit I (2023) Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools. *Queue* 20(6):Pages 10:35–Pages 10:57, DOI 10.1145/3582083, URL <https://dl.acm.org/doi/10.1145/3582083>
- Braun V, Clarke V (2012) Thematic analysis. In: *APA handbook of research methods in psychology*, Vol 2: Research designs: Quantitative, qualitative, neuropsychological, and biological, APA handbooks in psychology®, American Psychological Association, Washington, DC, US, pp 57–71, DOI 10.1037/13620-004
- Chen B, Jiang ZMJ (2022) A Survey of Software Log Instrumentation. *ACM Computing Surveys* 54(4):1–34, DOI 10.1145/3448976, URL <https://dl.acm.org/doi/10.1145/3448976>
- Chou YH (2026a) Manual check of collected logs through 100 commits sample across three languages. DOI 10.6084/m9.figshare.27325635, URL <https://figshare.com/s/3fc473024bed220aba75>

- Chou YH (2026b) Replication data (logs, commits, crawling and analysis script, and qualitative analysis on live stream). DOI 10.6084/m9.figshare.27638058, URL <https://doi.org/10.6084/m9.figshare.27638058>
- Chou YH, Min Y, Wang AY, Jones JA (2025) Learning from Mistakes: Understanding Ad-hoc Logs through Analyzing Accidental Commits. In: 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), pp 1–13, DOI 10.1109/MSR66628.2025.00017, URL <https://ieeexplore.ieee.org/abstract/document/11025683>
- Chou YH, Jiang B, Chen YW, Weng M, Jackson V, Zimmermann T, Jones JA (2026) Building software by rolling the dice: A qualitative study of vibe coding. *Proceedings of the ACM on Software Engineering* 3(FSE):1712–1734
- ChromeDevTools (2025) Google chrome minimized file detection algorithm. URL https://github.com/ChromeDevTools/devtools-frontend/blob/main/front_end/models/text_utils/TextUtils.ts#L336, accessed: 2024-11-09
- Ebert C, Cain J, Antoniol G, Counsell S, Laplante P (2016) Cyclo-matic Complexity. *IEEE Software* 33(6):27–29, DOI 10.1109/MS.2016.147, URL <https://ieeexplore.ieee.org/abstract/document/7725232>, conference Name: IEEE Software
- Foundation O (2025) Eslint for console.log. URL <https://eslint.org/docs/latest/rules/no-console>, accessed: 2024-11-09
- Fu Q, Zhu J, Hu W, Lou JG, Ding R, Lin Q, Zhang D, Xie T (2014) Where do developers log? an empirical study on logging practices in industry. In: Companion Proceedings of the 36th International Conference on Software Engineering, ACM, Hyderabad India, pp 24–33, DOI 10.1145/2591062.2591175, URL <https://dl.acm.org/doi/10.1145/2591062.2591175>
- Gholamian S, Ward PAS (2022) A Comprehensive Survey of Logging in Software: From Logging Statements Automation to Log Mining and Analysis. URL <http://arxiv.org/abs/2110.12489>, arXiv:2110.12489 [cs]
- Gilmore DJ (1991) Models of debugging. *Acta Psychologica* 78(1):151–172, DOI 10.1016/0001-6918(91)90009-O, URL <https://www.sciencedirect.com/science/article/pii/0001691891900090>
- GitHub, Inc (2025) Github rest api documentation (api version 2022-11-28). URL <https://docs.github.com/en/rest?apiVersion=2022-11-28>, accessed: 2025-09-30
- Google (2025) Google bigquery with github archive. URL <https://cloud.google.com/blog/topics/public-datasets/github-on-bigquery-analyze-all-the-open-source-code>, accessed: 2024-11-09
- Gordon AD, Negreanu C, Cambronerio J, Chakravarthy R, Drosos I, Fang H, Mitra B, Richardson H, Sarkar A, Simmons S, Williams J, Zorn B (2023) Co-audit: tools to help humans double-check AI-generated content. URL <http://arxiv.org/abs/2310.01297>, arXiv:2310.01297 [cs]
- Grigorik I (2015) Github archive. URL <https://www.gharchive.org/>, accessed: 2024-11-09

- Hao G, Hijazi H, Durães J, Medeiros J, Couceiro R, Lam CT, Teixeira C, Castelhana J, Castelo Branco M, Carvalho P, Madeira H (2023) On the accuracy of code complexity metrics: A neuroscience-based guideline for improvement. *Frontiers in Neuroscience* 16:1065366, DOI 10.3389/fnins.2022.1065366, URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC9942489/>
- Jiang P, Sun F, Xia H (2023) Log-it: Supporting Programming with Interactive, Contextual, Structured, and Visual Logs. In: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, ACM, Hamburg Germany, pp 1–16, DOI 10.1145/3544548.3581403, URL <https://dl.acm.org/doi/10.1145/3544548.3581403>
- Kalliamvakou E, Gousios G, Blincoe K, Singer L, German DM, Damian D (2014) The promises and perils of mining GitHub. In: *Proceedings of the 11th Working Conference on Mining Software Repositories*, Association for Computing Machinery, New York, NY, USA, MSR 2014, pp 92–101, DOI 10.1145/2597073.2597074, URL <https://dl.acm.org/doi/10.1145/2597073.2597074>
- Kernighan BW (1978) *UNIX For Beginners – Second Edition*
- Ko AJ, Myers BA, Coblenz MJ, Aung HH (2006) An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks. *IEEE Transactions on Software Engineering* 32(12):971–987, DOI 10.1109/TSE.2006.116, URL <https://ieeexplore.ieee.org/document/4016573>, conference Name: IEEE Transactions on Software Engineering
- Ko AJ, LaToza TD, Burnett MM (2015) A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Software Engineering* 20(1):110–141, DOI 10.1007/s10664-013-9279-3, URL <https://doi.org/10.1007/s10664-013-9279-3>
- Kokinda E, Rodeghero P (2023) Streaming software development: Accountability, community, and learning. *Journal of Systems and Software* 199:111630, DOI 10.1016/j.jss.2023.111630, URL <https://www.sciencedirect.com/science/article/pii/S0164121223000250>
- Lenarduzzi V, Kilamo T, Janes A (2023) Does Cyclomatic or Cognitive Complexity Better Represents Code Understandability? An Empirical Investigation on the Developers Perception. DOI 10.2139/ssrn.4397231, URL <https://papers.ssrn.com/abstract=4397231>
- Li H, Shang W, Hassan AE (2017) Which log level should developers choose for a new logging statement? *Empirical Software Engineering* 22(4):1684–1716, DOI 10.1007/s10664-016-9456-2, URL <https://doi.org/10.1007/s10664-016-9456-2>
- Li H, Shang W, Adams B, Sayagh M, Hassan AE (2021) A Qualitative Study of the Benefits and Costs of Logging From Developers’ Perspectives. *IEEE Transactions on Software Engineering* 47(12):2858–2873, DOI 10.1109/TSE.2020.2970422, URL <https://ieeexplore.ieee.org/abstract/document/8976297>, conference Name: IEEE Transactions on Software Engineering
- Li Z (2020) Towards providing automated supports to developers on writing logging statements. In: *Proceedings of the ACM/IEEE 42nd International*

- Conference on Software Engineering: Companion Proceedings, Association for Computing Machinery, New York, NY, USA, ICSE '20, pp 198–201, DOI 10.1145/3377812.3381385, URL <https://dl.acm.org/doi/10.1145/3377812.3381385>
- Li Z, Chen THP, Shang W (2020) Where shall we log?: studying and suggesting logging locations in code blocks. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, ACM, Virtual Event Australia, pp 361–372, DOI 10.1145/3324884.3416636, URL <https://dl.acm.org/doi/10.1145/3324884.3416636>
- Li Z, Chen AR, Hu X, Xia X, Chen TH, Shang W (2023) Are They All Good? Studying Practitioners’ Expectations on the Readability of Log Messages. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp 129–140, DOI 10.1109/ASE56229.2023.00136, URL <https://ieeexplore.ieee.org/abstract/document/10298380>
- Lieberman H (1997) The debugging scandal and what to do about it. Communications of the ACM 40(4):26–30
- Liu Z, Xia X, Lo D, Xing Z, Hassan AE, Li S (2021) Which Variables Should I Log? IEEE Transactions on Software Engineering 47(9):2012–2031, DOI 10.1109/TSE.2019.2941943, URL <https://ieeexplore.ieee.org/abstract/document/8840982>, conference Name: IEEE Transactions on Software Engineering
- Maalej W, Tiarks R, Roehm T, Koschke R (2014) On the Comprehension of Program Comprehension. ACM Trans Softw Eng Methodol 23(4):31:1–31:37, DOI 10.1145/2622669, URL <https://dl.acm.org/doi/10.1145/2622669>
- Mahtani K, Spencer EA, Brassey J, Heneghan C (2018) Catalogue of bias: observer bias. BMJ Evidence-Based Medicine 23(1):23–24, DOI 10.1136/ebmed-2017-110884, URL <https://ebm.bmj.com/content/23/1/23>
- Mastropaolo A, Pascarella L, Bavota G (2022) Using deep learning to generate complete log statements. In: Proceedings of the 44th International Conference on Software Engineering, Association for Computing Machinery, New York, NY, USA, ICSE '22, pp 2279–2290, DOI 10.1145/3510003.3511561, URL <https://dl.acm.org/doi/10.1145/3510003.3511561>
- Meli M, McNiece MR, Reaves B (2019) How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories. In: Proceedings 2019 Network and Distributed System Security Symposium, Internet Society, San Diego, CA, DOI 10.14722/ndss.2019.23418
- Rilling J, Klemola T (2003) Identifying comprehension bottlenecks using program slicing and cognitive complexity metrics. In: 11th IEEE International Workshop on Program Comprehension, 2003., pp 115–124, DOI 10.1109/WPC.2003.1199195, URL <https://ieeexplore.ieee.org/abstract/document/1199195>
- Scalabrino S, Bavota G, Vendome C, Linares-Vásquez M, Poshyvanyk D, Oliveto R (2017) Automatically assessing code understandability: How far are we? In: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), pp 417–427, DOI 10.1109/ASE.

- 2017.8115654, URL <https://ieeexplore.ieee.org/document/8115654/?arnumber=8115654>
- Shepperd M (1988) A critique of cyclomatic complexity as a software metric. *Software Engineering Journal* 3(2):30–36, DOI 10.1049/sej.1988.0003, URL <https://digital-library.theiet.org/doi/abs/10.1049/sej.1988.0003>
- Siegmund B, Perscheid M, Taeumel M, Hirschfeld R (2014) Studying the Advancement in Debugging Practice of Professional Software Developers. In: 2014 IEEE International Symposium on Software Reliability Engineering Workshops, pp 269–274, DOI 10.1109/ISSREW.2014.36, URL <https://ieeexplore.ieee.org/document/6983851>
- Trümper J, Bohnet J, Döllner J (2010) Understanding complex multithreaded software systems by using trace visualization. In: Proceedings of the 5th international symposium on Software visualization, Association for Computing Machinery, New York, NY, USA, SOFTVIS '10, pp 133–142, DOI 10.1145/1879211.1879232, URL <https://dl.acm.org/doi/10.1145/1879211.1879232>
- Turcotte A, Shah MD, Aldrich MW, Tip F (2022) DrAsync: identifying and visualizing anti-patterns in asynchronous JavaScript. In: Proceedings of the 44th International Conference on Software Engineering, Association for Computing Machinery, New York, NY, USA, ICSE '22, pp 774–785, DOI 10.1145/3510003.3510097, URL <https://dl.acm.org/doi/10.1145/3510003.3510097>
- Wen F, Nagy C, Lanza M, Bavota G (2020) An Empirical Study of Quick Remedy Commits. In: Proceedings of the 28th International Conference on Program Comprehension, Association for Computing Machinery, New York, NY, USA, ICPC '20, pp 60–71, DOI 10.1145/3387904.3389266, URL <https://dl.acm.org/doi/10.1145/3387904.3389266>
- Xia X, Bao L, Lo D, Xing Z, Hassan AE, Li S (2018) Measuring Program Comprehension: A Large-Scale Field Study with Professionals. *IEEE Transactions on Software Engineering* 44(10):951–976, DOI 10.1109/TSE.2017.2734091, URL <https://ieeexplore.ieee.org/abstract/document/7997917>
- Yang N, Cuijpers P, Schiffelers R, Lukkien J, Serebrenik A (2021) An Interview Study of how Developers use Execution Logs in Embedded Software Engineering. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp 61–70, DOI 10.1109/ICSE-SEIP52600.2021.00015, URL <https://ieeexplore.ieee.org/abstract/document/9402136>
- Yuan D, Park S, Zhou Y (2012) Characterizing logging practices in open-source software. In: Proceedings of the 34th International Conference on Software Engineering, IEEE Press, Zurich, Switzerland, ICSE '12, pp 102–112
- Zhu J, He P, Fu Q, Zhang H, Lyu MR, Zhang D (2015) Learning to Log: Helping Developers Make Informed Logging Decisions. In: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, vol 1, pp 415–425, DOI 10.1109/ICSE.2015.60, URL <https://ieeexplore.ieee.org/>

```
document/7194593/?arnumber=7194593
```